A Nonreflecting Boundary Condition for the Heat Equation in One Dimension

JunHyung Chang

This is a final project for Numerical Methods II.
The full m-file is attached separately.

Overview

In order to solve the heat equation on $\mathbb{R}$ numerically, we must find certain boundary conditions for the numerical boundaries that simulate the problem on the whole real line. In this project, we use the Crank-Nicholson method to solve the heat equation on $[0,1]$ with compactly supported Gaussian initial data. The goal is to derive, and implement non-reflecting boundary conditions for the interval $[0,1]$. The main task is designing a quadrature rule for the integral that appears in the expression for the boundary conditions.

1. The boundary condition

Consider the linear PDE

$$u_t = u_{xx}, \quad u(x,0) = \frac{1}{\sqrt{4\pi t_0}} e^{-(x-0.5)^2/4t_0}, \quad x \in \mathbb{R}, \quad t \in [0,1],$$

where we take t_0 small so that $u(x,0)$ disappears outside $[0,1]$. Since the initial data is zero outside $[0,1]$, the problem reduces to solving the heat equation with zero initial data on $[1,\infty)$ and $(-\infty,0]$. Focusing on $[1,\infty)$, we can view the problem as a Neumann problem by approximating $u_x(1,t)$ using the data. Thus, the solution on $[0,\infty)$ in terms of the Green's function for the heat equation is

$$\begin{aligned} u(x,t) &= \int_0^t G(x-1,t-\tau) f(\tau) d\tau \\ &= \int_0^t \frac{e^{-(x-1)^2/4(t-\tau)}}{\sqrt{4\pi(t-\tau)}} f(\tau) d\tau, \end{aligned} \tag{1}$$

where $f(t)$ is an unknown density. Viewing $G(x-1,t)$ as an approximate delta function centered at $x=1$, we observe that

$$\left.\frac{\partial u(x,t)}{\partial x}\right|_{x=1} := u_x(1,t) = -\frac{1}{2} f(t). \tag{2}$$

Plugging (2) back into (1), and repeating this in the analogous case for $x=0$, we obtain the nonreflecting boundary conditions

$$u(0,t) = \frac{1}{\sqrt{\pi}} \int_0^t \frac{u_x(0,\tau)}{\sqrt{t-\tau}} d\tau, \quad u(1,t) = -\frac{1}{\sqrt{\pi}} \int_0^t \frac{u_x(1,\tau)}{\sqrt{t-\tau}} d\tau. \tag{3}$$

We now devise a quadrature rule to evaluate these integrals.

2. Evaluating boundary conditions

We implement the method introduced in the paper by Jiang and Greengard [1]. It is difficult to come up with a direct quadrature rule for (3) due to the square root singularity at t. The authors overcome this difficulty by dividing the integrals into a local part, I_{loc}, and a history part I_{hist}, where

$$\begin{aligned} I_{loc,0}(n) &= \frac{1}{\sqrt{\pi}} \int_{t-\Delta t}^{t} \frac{u_x(0,\tau)}{\sqrt{t-\tau}} d\tau, \quad (t = n\Delta t) \\ I_{hist,0}(n) &= \frac{1}{\sqrt{\pi}} \int_{0}^{t-\Delta t} \frac{u_x(0,\tau)}{\sqrt{t-\tau}} d\tau. \end{aligned} \tag{4}$$

Note that the boundary condition for $x = 1$ is symmetrical. The idea is to compute I_{hist} as a convolution of $u_x(0,\tau)$ with a sum of exponentials approximation of the form

$$\frac{1}{\sqrt{\pi t}} = \sum_{j=1}^{N} w_j e^{-s_j^2 t},$$

and to compute I_{loc} by viewing $u_x(0,\tau)$ inside the integral as a linear interpolating function.

To use the sum of exponentials approximation in the calculation for I_{hist}, we must be able to control the error to an arbitrary precision ϵ by choosing appropriate nodes and weights. In [1, lemma 5, lemma 6, theorem 7], the authors split the interval into dyadic sub-intervals $[2^j, 2^{j+1}]$, and take $s_{j,1}, \ldots, s_{j,n}$ and $w_{j,1}, \ldots, w_{j,n}$, which are the n-point Gaussian quadrature nodes and weights for each sub-interval. For the first interval $[0, 2^{Lmin}]$, the authors take $s_{0,1}, \ldots, s_{0,n_0}$ and $w_{0,1}, \ldots, w_{0,n_0}$, the n_0-point Gaussian quadrature nodes and weights. Leaving out details, it is stated in [1, theorem 7] that if we take

$$\begin{aligned} n &= \left\lceil \frac{1}{3} \log_2 \left[\frac{12(L_{max} - L_{min} + 1)}{\epsilon} \right] \right\rceil, \\ n_0 &= \lceil 0.565 \log(10/\epsilon) \rceil, \\ L_{min} &= \left\lfloor \log_2 \sqrt{n_0/T} \right\rfloor, \\ L_{max} &= \left\lfloor \log_2 \sqrt{(\log 1/\epsilon)/\delta} \right\rfloor, \end{aligned}$$

then

$$\frac{2}{\pi} \left(\sum_{k=1}^{n_0} w_{0,k} e^{-s_j^2 t} + \sum_{j=L_{min}}^{L_{max}} \sum_{k=1}^{n} w_{j,k} e^{-s_{j,k}^2 t} \right) \tag{5}$$

approximates $1/\sqrt{\pi t}$ to the precision $O(\epsilon)$. A quick numerical experiment shows that n and n_0 required to reach machine precision is very reasonable, hence we choose $\epsilon = O(10^{-16})$ and do not worry about any errors at this stage.

Using this result, we compute I_{hist} by substituting $1/\sqrt{\pi(t-\tau)}$ with the expression above using $t - \tau$ instead of t. Namely,

$$I_{hist}(n) = \sum_{j=1}^{N} w_j C_j(n),$$

where

$$C_j(n) = \int_0^{(n-1)\Delta t} e^{-s_j^2 (n\Delta t - \tau)} u_x(0,\tau) d\tau.$$

Instead of deriving another quadrature rule for each $C_j(n)$, which would be very burdensome in terms of memory and flops, the authors make use of the recursive formula

$$C_j(n) = e^{-s_j^2 \Delta t} C_j(n-1) + \int_{(n-2)\Delta t}^{(n-1)\Delta t} e^{-s_j^2 (n\Delta t - \tau)} u_x(0,\tau) d\tau. \tag{6}$$

This greatly reduces computational burden, since we do not need to store history values of the integrand anymore. In order to compute the integral in (6), we approximate $u_x(0,\tau)$ inside the integral with a linear interpolating function

$$\sigma(s) = u_x(0,(n-2)\Delta t) + \frac{u_x(0,(n-1)\Delta t) - u_x(0,(n-2)\Delta t)}{k}(s-(n-2)k), \tag{7}$$

where we use a one sided second-order approximation for $u_x(0, n\Delta t)$

$$u_x(0, n\Delta t) \approx \frac{1}{2h}(-3U_0^n + 4U_1^n - U_2^n). \tag{8}$$

Note that the approximation of $u_x(1, n\Delta t)$ will be

$$u_x(1, n\Delta t) \approx \frac{1}{2h}(3U_{m+1}^n - 4U_m^n + U_{m-1}^n). \tag{9}$$

Plugging (7) into (6), and performing integration by parts, we obtain a closed-form expression for each $C_j(n)$, which can be evaluated using (8) and (9). Namely,

$$\begin{aligned} C_j(n) = {} & e^{-s_j^2 \Delta t} C_j(n-1) \\ & + \frac{1}{s_j^2}(e^{-s_j^2 \Delta t} - e^{-2s_j^2 \Delta t})\Big[u_x(0,(n-2)k) - (n-2)(u_x(0,(n-1)k) - u_x(0,(n-2)k))\Big] \\ & + \frac{1}{s_j^2}((n-1)ke^{-s_j^2 \Delta t} - (n-2)ke^{-2s_j^2 \Delta t}) - \frac{1}{s_j^4}(e^{-s_j^2 \Delta t} - e^{-2s_j^2 \Delta t}). \end{aligned}$$

Thus, $I_{hist}(n)$ can now be computed by repeating this for each dyadic interval. One must be careful not to forget the factor of $2/\pi$ in (5).

To compute I_{loc}, the authors of [1] use an approximation for $u_x(0,\tau)$ on the interval $[t-\Delta t, t]$ similar to (7). It is stated in [1, lemma 9] that

$$I_{loc,0}(n) \approx \sqrt{\frac{\Delta t}{\pi}}\Big(\frac{4}{3}u_x(0, n\Delta t) + \frac{2}{3}u_x(0,(n-1)\Delta t)\Big). \tag{10}$$

3. Implementing the solver

The m-file `heat_CN_nrbc.m` in the codes zip file modifies the code `heat_CN.m` from homework 5. We use $\Delta t = k = h = 1/(m+1)$, $T =$`tfinal`$= 1$, and $t_0 = 10^{-3}$ in this experiment. `heat_CN_nrbc.m` uses `legpts` in the Chebfun package to obtain the Gaussian quadrature nodes and weights.

First, note that the approximation for $I_{loc}(n)$ in (10) requires data from time $n\Delta t$. Thus, we must include I_{loc} implicitly in the time-stepping matrix alongside the Crank-Nicholson scheme.

That is, using (8), (9) and (10), the time-stepping system of equations will be

$$\begin{bmatrix} 1+6q & -8q & 2q & & & \\ -r & 1+2r & -r & & & \\ & & \ddots & & & \\ & & -r & 1+2r & -r \\ & & 2q & -8q & 1+6q \end{bmatrix} \begin{bmatrix} U_0^{n+1} \\ \vdots \\ U_{m+2}^{n+1} \end{bmatrix} = \begin{bmatrix} -3q & 4q & -q & & & \\ -r & 1+2r & -r & & & \\ & & \ddots & & & \\ & & -r & 1+2r & -r \\ & & -q & 4q & -3q \end{bmatrix} \begin{bmatrix} U_0^{n} \\ \vdots \\ U_{m+2}^{n} \end{bmatrix} + \begin{bmatrix} I_{hist,0} \\ 0 \\ \vdots \\ 0 \\ -I_{hist,1} \end{bmatrix},$$

where $r = k/(2h^2)$ arises from the Crank-Nicholson scheme, and $q = \sqrt{\Delta t/\pi}/(3h)$ arises from the computation of I_{loc} in (10). Namely, the first and last equations are designed to solve

$$U(0,(n+1)t) = I_{loc,0}(n+1) + I_{hist,0}(n+1),$$

$$U(1,(n+1)t) = -(I_{loc,1}(n+1) + I_{hist,1}(n+1)).$$

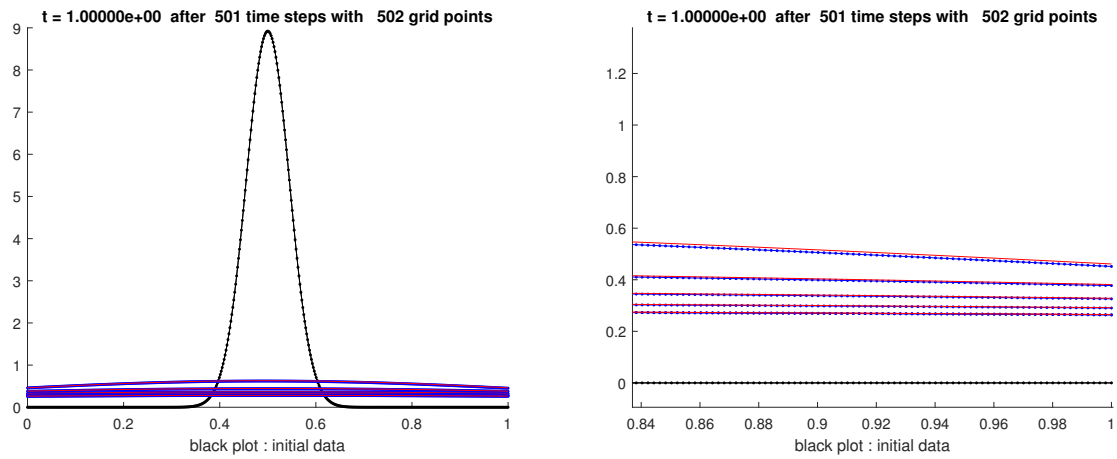
4. Results

Since the analytic solution of the heat equation on $\mathbb{R}$ is known, we can compute the max errors of the computed solution for different Δt. As in the homework problems, we test the accuracy of the method by using the code `error_table.m`. This is done in the m-file `errortest.m` in the codes zip file. The order of accuracy may be improved by using a higher order interpolating function in (7) and (10), or if the Neumann boundary data is provided explicitly. The accuracy of this method is summarized in the table below.

Table 1. the max error and observed order for decreasing Δt.

grid points	Δt	max error	ratio	obs. order
100	0.00990	4.56752e-03	`NaN`	`NaN`
500	0.00200	2.03488e-03	2.24461	0.50486
1000	0.00100	1.43822e-03	1.41486	0.50138
2000	0.00050	1.01675e-03	1.41453	0.50068
4000	0.00025	7.18871e-04	1.41437	0.50034

Figure 1. Plot of solutions and a close-up view near $x = 1$.



References

[1] S. Jiang, L. Greengard. *Fast Evaluation of Nonreflecting Boundary Conditions for the Schrödinger Equation in One Dimension.* Computers and Mathematics with Applications, 47, 955-966, 2004.

[2] A. Ditkowski, A. Suhov. *Near-field infinity-simulating boundary conditions for the heat equation.* Proceedings of the National Academy of Sciences, 105 (31) 10646-10648, Aug 2008.

[3] T. Hagstrom. *Radiation boundary conditions for the numerical simulation of waves.* Acta Numerica, 8, 47-106, 1999.